



MÁSTER EN INTELIGENCIA ARTIFICIAL · INFORME DE PROYECTO

UrbanPlan AI

Sistema Inteligente de Consultoría Urbanística basado en Arquitectura Multi-Agente

Asignatura: MLOPs

Profesor: [Daniel Ruiz Riquelme](#)

Programa: Máster en Inteligencia Artificial (MIA) — EDEM

Integrantes del grupo

- Fátima Cucart Matoses
- Florencia Sastre
- Carlos Sevillano
- Alejandro Moreno Sapena
- Raul Aragall Taberner

Índice

1. Escenario y Visión	3
2. Arquitectura del Sistema	3
3. Desglose de Responsabilidades	4
4. Componentes Técnicos Detallados	4
4.1. Supervisor (supervisor_urbanplan)	4
4.2. Agente Normativo (agente_normativo)	5
4.3. Agente Estimador (agente_estimador)	5
4.4. Herramienta de Dossier y Servidor MCP	6
5. Tecnologías y Prácticas MLOps	6
6. Flujo de Ejecución y Casos de Uso	6
7. Seguridad y Control de Coste	7
8. Modos de Fallo y Limitaciones	7
9. Estado de Implementación y Líneas Futuras	8

1. Escenario y Visión

UrbanPlan AI es un sistema multiagente cuyo objetivo es facilitar el acceso a la información urbanística y de licencias de obra. El usuario plantea en lenguaje natural una consulta sobre cualquier tipo de actuación (reforma, obra mayor, demolición, etc.) y el sistema responde determinando dos dimensiones del proyecto:

- **Viabilidad legal y normativa:** resuelta por un agente experto que se apoya en el conocimiento del modelo de lenguaje sobre el Código Técnico de la Edificación (CTE) y la normativa habitual.
- **Plazos y costes estimados:** resueltos por un agente que consulta una base de datos histórica en Google BigQuery, generando dinámicamente el SQL necesario para que los cálculos se basen en datos y no en estimaciones inventadas.

Cuando el usuario lo solicita, el sistema compila ambas respuestas en un **dossier ejecutivo** en formato Markdown, que se guarda en disco mediante una herramienta dedicada.

Alcance de este informe. Este documento describe el sistema **tal y como está implementado en el código entregado**. Las funcionalidades planificadas pero no integradas (p. ej. recuperación vectorial RAG, evaluación automatizada o despliegue real en Cloud Run) se recogen de forma explícita en la sección 9 “Estado de implementación y líneas futuras”.

2. Arquitectura del Sistema

El sistema implementa el patrón **Supervisor / Sub-agente** de Google ADK. Un agente supervisor recibe la consulta del usuario y, mediante el mecanismo nativo de transferencia de ADK (`transfer_to_agent`), decide de forma semántica a qué especialista delegar. No existe un enrutado por reglas o palabras clave codificado a mano: la clasificación de intención la realiza el propio LLM a partir de las descripciones e instrucciones de cada agente.

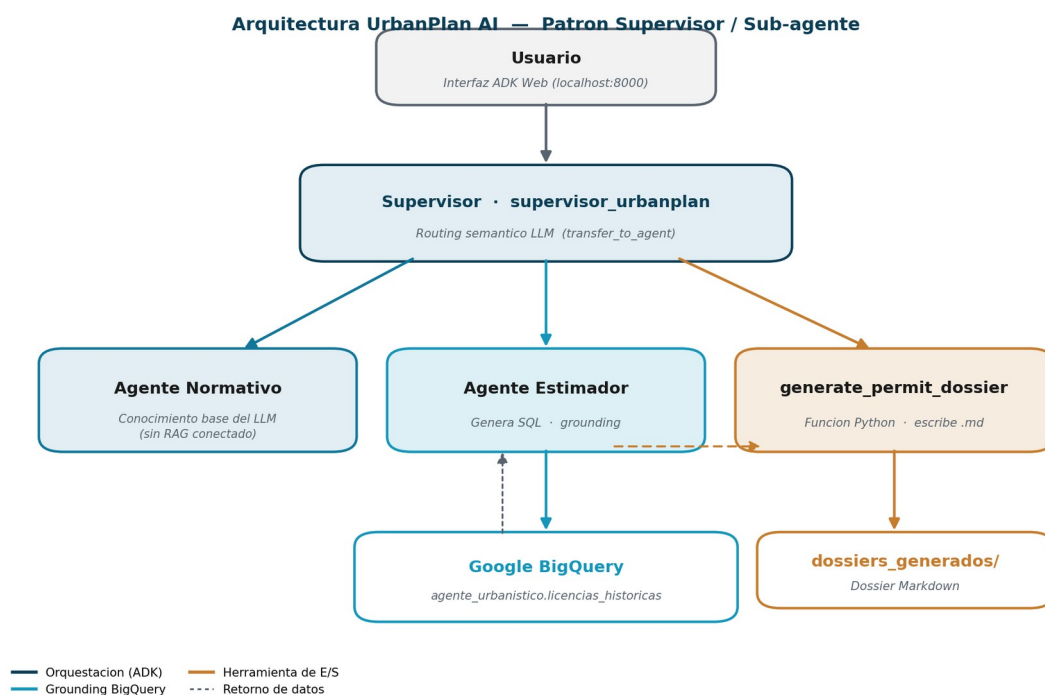


Figura 1. Flujo de orquestación y componentes del sistema implementado.

El flujo de información es el siguiente:

1. El **Usuario** interactúa a través de la interfaz web de ADK (adk web src, en localhost:8000).
2. El **Supervisor** (supervisor_urbanplan) analiza la intención y transfiere la conversación al sub-agente adecuado, o invoca directamente la herramienta de dossier.
3. El **Agente Normativo** responde sobre viabilidad legal con el conocimiento del modelo.
4. El **Agente Estimador** genera y ejecuta SQL contra BigQuery para obtener medias de tasas y plazos a partir del histórico.
5. La herramienta generate_permit_dossier compone el informe final y lo escribe como archivo .md en dossiers_generados/.

3. Desglose de Responsabilidades

El desarrollo se reparte en tres perfiles, alineados con los componentes realmente presentes en el repositorio:

Perfil	Rol	Responsabilidades técnicas (sobre el código)
Persona A — Arquitecto de Agentes	Lógica y orquestación	Definición del supervisor y los sub-agentes, redacción de las instrucciones (system prompts), encadenamiento mediante sub_agents y guardrails de seguridad.
Persona B — Ingeniería de Datos	BigQuery y datos	Script de creación y población del dataset, clase de acceso a BigQuery con validación de seguridad y control de coste, diseño del esquema de la tabla histórica.
Persona C — Backend / DevOps	Herramientas y despliegue	Herramienta de generación de dossier y servidor MCP, contenerización con Docker, configuración de entorno y pipeline de CI/CD en GitHub Actions.

4. Componentes Técnicos Detallados

4.1. Supervisor (supervisor_urbanplan)

Definido en src/urbanplan/supervisor/supervisor.py. Es un Agent de ADK que instancia a los dos sub-agentes y registra la herramienta de dossier. Su instrucción describe el equipo disponible y el flujo a seguir para producir un dossier completo (primero normativa, luego estimación, finalmente generación del archivo).

- **Entrada:** prompt del usuario en lenguaje natural.
- **Salida:** transferencia a agente_normativo o agente_estimador, o llamada a generate_permit_dossier.
- **Mecanismo:** delegación semántica vía sub_agents (no hay enrutado por reglas).
- **Guardrails:** la instrucción incluye una sección de seguridad que bloquea insultos, discurso de odio, consejos para eludir tasas o saltarse el CTE, e intentos de jailbreak.

4.2. Agente Normativo (agente_normativo)

Definido en `src/urbanplan/agents/normativo.py`. Es un agente **sin herramientas**: responde a consultas de viabilidad legal apoyándose exclusivamente en el conocimiento del modelo `gemini-2.5-flash` sobre el CTE y la normativa habitual. Su instrucción le indica actuar como arquitecto experto y responder de forma profesional y concisa.

Sobre el RAG. El repositorio incluye un notebook (`notebooks/indexacion_normativa.ipynb`) que prepara un índice vectorial FAISS con LlamaIndex a partir de PDFs. Sin embargo, en el estado actual del código **ese índice no está conectado al agente** y no existen documentos de entrada ni índice generado en el repositorio. Por tanto, el Agente Normativo funciona con el conocimiento del LLM; el RAG figura como línea futura (sección 9).

4.3. Agente Estimador (agente_estimador)

Definido en `src/urbanplan/agents/estimador.py`. Es el componente que aporta **grounding** sobre datos reales. Dispone de dos herramientas:

- `query_historical_data`: ejecuta una consulta SQL de solo lectura contra BigQuery y devuelve hasta 20 filas formateadas como texto al modelo.
- `generate_permit_dossier`: misma herramienta de dossier disponible también para el supervisor.

Su instrucción contiene reglas de generación de SQL pensadas para obtener resultados útiles del histórico:

- Consultar siempre la tabla histórica antes de responder sobre costes o plazos.
- Nunca filtrar por igualdad exacta (`=`); usar rangos amplios con `BETWEEN` para encontrar obras similares.
- Usar comodines `LIKE '%...%'` cuando no se conozca el tipo de obra exacto.
- Si la consulta falla, ofrecer una estimación basada en conocimiento del sector (comportamiento de *fallback*).

La tabla objetivo es `agente_urbanistico.licencias_historicas`, ubicada en el proyecto de GCP configurado en el entorno, con columnas: `tipo_obra`, `municipio`, `presupuesto_euros`, `tasa_licencia_euros`, `dias_tramitacion`, `metros_cuadrados` y `anio`.

Capa de acceso a datos: BigQueryEstimatorTool

Definida en `src/urbanplan/retrieval/bigquery.py`. Encapsula el cliente de BigQuery (región EU) y aplica una capa de seguridad antes de ejecutar cualquier consulta:

- Solo se admiten sentencias que empiecen por `SELECT` (validación por expresión regular).
- Se bloquean palabras clave de modificación: `INSERT`, `UPDATE`, `DELETE`, `DROP`, `TRUNCATE`, `ALTER`.
- Si la consulta no incluye `LIMIT`, se le añade `LIMIT 1000` automáticamente.
- Se fija un límite de coste con `maximum_bytes_billed = 100 MB` por consulta.
- Los errores se capturan y se devuelven como dato, evitando que una consulta fallida tumbe el agente.

4.4. Herramienta de Dossier y Servidor MCP

La función `generate_permit_dossier` (`src/urbanplan/mcp_server/dossier_tool.py`) recibe el título del proyecto, el resumen de viabilidad legal, el coste estimado y los días de tramitación, y escribe un dossier en Markdown en la carpeta `dossiers_generados/`, devolviendo la ruta del archivo generado.

El proyecto incluye además un **servidor MCP independiente** (`server.py`) construido con FastMCP, que expone la misma operación como herramienta MCP en modo `stdio`. En la integración con el agente, sin embargo, se utiliza la versión como función Python por compatibilidad; el servidor MCP queda disponible como interfaz alternativa.

Formato de salida. El dossier se genera en **Markdown (.md)**. El código entregado no produce salida en JSON.

5. Tecnologías y Prácticas MLOps

Área	Implementación en el repositorio
Gestión de entorno	<code>uv</code> con instalación determinista a partir de <code>uv.lock</code> (<code>uv sync --frozen</code>). Dependencias declaradas en <code>pyproject.toml</code> .
Modelo	<code>gemini-2.5-flash</code> vía Vertex AI, configurado por variables de entorno.
Data engineering	<code>setup_bigquery.py</code> crea la tabla histórica e inyecta 100 registros sintéticos realistas (10 tipos de obra, 10 municipios, años 2018-2024) con lógica diferenciada por categoría.
Contenerización	Dockerfile sobre <code>python:3.12-slim</code> , instala dependencias con <code>uv</code> , expone el puerto 8080 y arranca <code>adk web</code> .
CI/CD	GitHub Actions (<code>.github/workflows/ci.yml</code>). CI: checkout, instalación con <code>uv</code> , <code>uv pip check</code> y verificación de sintaxis con <code>py_compile</code> . CD: build de la imagen Docker y despliegue a Cloud Run simulado (paso informativo).

Configuración de entorno necesaria (archivo `.env`):

```
GOOGLE_CLOUD_PROJECT=<id-proyecto-gcp>
GOOGLE_CLOUD_LOCATION=europe-west1
GOOGLE_GENAI_USE_VERTEXAI=True
GEMINI_MODEL=gemini-2.5-flash
```

6. Flujo de Ejecución y Casos de Uso

El sistema se valida mediante tres consultas, cada una ejercitando un componente distinto:

Caso 1 — Conocimiento normativo (Agente Normativo)

“¿Cuáles son los requisitos de altura libre mínima en viviendas según el CTE?” El supervisor transfiere al agente normativo, que responde con conocimiento del modelo.

Caso 2 — Consulta SQL autónoma (Agente Estimador)

“Basándote en el histórico, ¿cuál es la tasa media de licencia y los días de tramitación para una obra mayor residencial en Madrid?” El estimador genera una consulta de agregación (AVG) con filtros por rango y la ejecuta sobre BigQuery.

Caso 3 — Generación de dossier (Herramienta)

“Calcula el presupuesto de una obra de local en Valencia de 200 m² y génrame un dossier en disco con los resultados.” El sistema combina estimación y normativa y escribe un archivo .md en `dossiers_generados/`.

La interfaz de ADK permite inspeccionar la traza (*Trace*), donde se observa la secuencia `transfer_to_agent` → `query_historical_data`, confirmando que el sistema ejecuta herramientas reales y no improvisa.

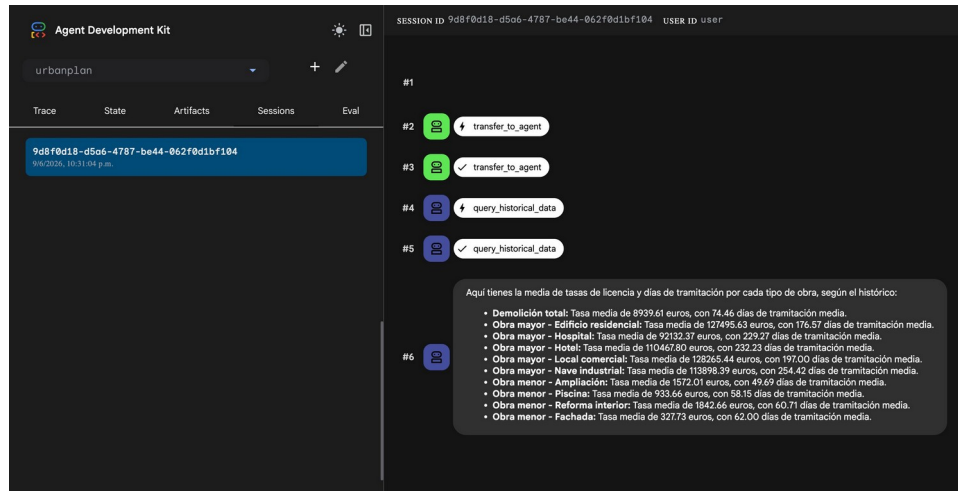


Figura 2. Traza de ADK: el supervisor delega en el agente estimador, que consulta BigQuery y devuelve medias reales de tasas y plazos por tipo de obra.

7. Seguridad y Control de Coste

Defensa de la base de datos

- Solo lectura: validación de prefijo `SELECT` y lista negra de comandos de modificación.
- Límite de filas automático (`LIMIT 1000`) y de coste (`maximum_bytes_billed = 100 MB`).
- Captura de errores para degradación controlada.

Defensa del agente

- Guardrails en la instrucción del supervisor contra contenido ofensivo, asesoramiento ilegal e intentos de jailbreak.
- El agente debe ceñirse a la legalidad urbanística y rechazar peticiones que busquen eludir tasas o el CTE.

8. Modos de Fallo y Limitaciones

Riesgo / limitación	Comportamiento actual y mitigación
Consulta a BigQuery fallida	El estimador recurre a una estimación basada en conocimiento del sector (fallback). Útil para no romper la conversación, pero pierde el grounding; conviene vigilar que no se confunda con un dato real.
Normativa fuera del conocimiento del LLM	Al no haber RAG conectado, el Agente Normativo puede no cubrir normativa local muy específica (PGOU municipales).

Riesgo / limitación	Comportamiento actual y mitigación
	Riesgo de respuesta imprecisa; mitigable integrando el índice vectorial del notebook.
Latencia	Las consultas a BigQuery más las llamadas al LLM añaden latencia perceptible en flujos largos (dossier completo).
Prompt injection	Mitigado parcialmente por los guardrails del supervisor y por la validación estricta de SQL en la capa de datos.
Coste	Acotado en BigQuery por el límite de bytes facturables; el consumo de tokens del LLM no tiene tope explícito en el código.

9. Estado de Implementación y Líneas Futuras

Resumen honesto de qué está operativo en el código entregado y qué queda como evolución:

Funcionalidad	Estado	Observaciones
Orquestación supervisor + sub-agentes (ADK)	Implementado	Funcional con routing semántico.
Agente Estimador con grounding en BigQuery	Implementado	Genera y ejecuta SQL real.
Herramienta de dossier (Markdown)	Implementado	Escribe .md en disco.
Servidor MCP independiente (stdio)	Implementado	Disponible; no es la ruta por defecto.
CI/CD con GitHub Actions	Implementado	CD de Cloud Run en modo simulado.
RAG normativo (FAISS + LlamaIndex)	Prototipo	Notebook sin datos ni conexión al agente.
Exportación del dossier en JSON	Pendiente	Solo Markdown actualmente.
Evaluación automatizada (DeepEval / Golden Dataset)	Pendiente	No presente en el repositorio.
Cacheo (Redis) para latencia	Pendiente	Propuesta de mejora.
Despliegue real en Cloud Run	Pendiente	El paso de CD solo simula el despliegue.

Priorización sugerida para una siguiente iteración:

1. Conectar el índice FAISS al Agente Normativo para sustituir conocimiento genérico por normativa indexada (mayor impacto en precisión).
2. Añadir un conjunto de evaluación (ToolCorrectness) para medir si el supervisor delega correctamente.
3. Activar el despliegue real en Cloud Run en el pipeline de CD.